

PROJE 15 / 20 · ÇALIŞAN TEMEL SÜRÜM

Çok Dilli Mini LMS

Ders içeriklerini iki dilde göster; kullanıcı ilerlemesini idempotent biçimde kaydet.

Türkçe/İngilizce · üç ders

Bu proje nedir?

Frontend kullanıcının gördüğü ekranları ve etkileşimleri, backend iş kurallarını ve veri erişimini yürütür. Bu uygulama ikisini aynı origin üzerinde birleştiren bağımsız bir full-stack örnektir. Üretim ürünü değil, üzerinde geliştirme yapabileceğin çalışan başlangıçtır.

DYFULL-STACK LAB / 15 / 20

Deniz DemoÇıkış

PORTFOLYO PROJESİ · ÇALIŞAN TEMEL SÜRÜM

Çok Dilli Mini LMS

Ders içeriklerini iki dilde göster; kullanıcı ilerlemesini idempotent biçimde kaydet.

Türkçe/İngilizce · üç ders

İşlem tamamlandı

Dil / Language

Türkçe

Web Geliştirme

Üç kısa dersle istemci, sunucu ve veri ilişkisi.

33% tamamlandı

HTTP ve API

İstemci bir istek gönderir; sunucu durum kodu ve içerik döndürür. 200 başarı, 404 bulunamadı anlamına gelir.

Tamamlanmadı olarak işaretli

Gerçek uygulama ekranı · yerel demo verileri

Teslim edilen kapsam

Dil seçimi · Çeviri tablosu · Türkçeye fallback · Tamamlandı/geri al · Yüzde ilerleme

Bu sürümde olmayanlar

Sabit bir kurs ve üç kısa metin dersi. Yönetici içerik editörü, video, sınav, sertifika ve ücretli kayıt yok. Eksik İngilizce ders Türkçe gösterilir.

02 / Backend ve veri modeli

Çeviriler (entity,locale) birleşik anahtarındadır. Seçilen dil bulunamazsa Türkçe okunur ve arayüz fallback uyarısı gösterir. progress (owner,lesson) anahtarı aynı dersin iki kez sayılmasını engeller. Tamamlanma PUT ile son durumu yazar; tekrar istek ilerlemeyi şişirmez.

Gerçek SQL şeması

```
CREATE TABLE courses(id TEXT PRIMARY KEY);
```

```
CREATE TABLE lessons(id TEXT PRIMARY KEY,course TEXT REFERENCES courses(id),position INTEGER);
```

```
CREATE TABLE translations(entity TEXT,locale TEXT,title TEXT,body TEXT,PRIMARY KEY(entity,locale));
```

```
CREATE TABLE progress(owner TEXT REFERENCES users(id),lesson TEXT REFERENCES lessons(id),completed INTEGER,PRIMARY KEY(owner,lesson));
```

Tam şema: schema.sql. users ve sessions ortak kimlik tablolarıdır. owner/user alanları kullanıcıyı, PRIMARY KEY kaydı, UNIQUE tekilliği ve REFERENCES ilişkisi belirtir. Para kullanılan projelerde INTEGER kuruş, zaman alanlarında ISO metin veya epoch kullanılır.

Kod okuma sırası

Dosya	Ne arayacaksın?
client/Workspace.vue	Eylemi tetikleyen form ve API isteği
client/api.ts	JSON isteği, hata yakalama ve mesaj
server/project.js	Yetki kontrolü → kural → SQL → yanıt
server/core.js	Parametrelili SQL, transaction, oturum, SSE

Neden bu mimari?

Aynı dilde frontend/backend öğrenmeyi kolaylaştırır. Node.js 24 ve Express HTTP katmanıdır; Vue arayüzü yönetir. SQLite ayrıca sunucu kurmadan kalıcılık sağlar. Senkron veritabanı işlemleri küçük yerel demo için seçildi; çok süreçli, yüksek trafikli üretim mimarisi iddiası yok.

03 / API sözleşmesi

Aşağıdaki yollar doğrudan teslim edilen server/project.js dosyasından çıkarılmıştır. JSON alan adları kodla aynıdır. :id ve benzeri path alanları oluşturma yanıtından alınmalıdır. Liste/görünüm uçlarında 200, yeni kayıt uçlarında 201, kuyruk kabulünde 202 kullanılır.

Yöntem / yol	Girdi alanları	Başarı
GET /api/courses	Query: locale	200
PUT /api/progress/:lesson	JSON: completed	200

Somut istek örneği

```
PUT /api/progress/l1
{"completed":true}
200 {"ok":true}
```

Oturum ve hata biçimi

POST /api/auth/login: email ve password. Başarılı giriş HttpOnly çerez verir. GET /api/auth/me kullanıcıyı, POST /api/auth/logout çıkışı yönetir. Tarayıcı aynı origin kullanır. Genel hata biçimi: {"error":"açıklama"}. Yetkisiz alan işlemleri sunucuda engellenir; frontend kontrolü tek başına yeterli değildir.

Kod	Anlam
401 / 403	Giriş gerekli / rol veya origin reddedildi
404 / 409	Kayıt görünmüyor / durum veya benzersizlik çakışması
422 / 429	Geçersiz girdi / giriş veya uygulama kotası
500 / 502 / 503	Beklenmeyen hata / dış servis hatası / kapalı veya ayarsız servis

Tüm uçlar bütün kodları üretmez. İstek/yanıt gövdelerini tests/project.test.js ve scripts/seed.js içindeki çalışan örneklerle karşılaştır.

04 / Test ve kabul kriterleri

Bir projenin çalışması yalnız ekrana veri gelmesi değildir. Yetki, tekrar istek, yanlış girdi ve yarış koşulları da beklenen sonucu üretmelidir. Aşağıdaki maddeler teslim edilen temel sürümün kontrol listesidir.

- İngilizce seçildiğinde İngilizce başlık gelmeli.
- Eksik ders çevirisi Türkçeye dönmeli.
- Aynı dersi iki kez tamamlamak tek kayıt sayılmalı.
- Diğer kullanıcının ilerlemesi değişmemeli.

Otomatik test nasıl çalışır?

npm test, Node test runner ile gerçek HTTP sunucusunu rastgele yerel portta başlatır. İki kullanıcıyla oturum açar, uç noktalara istek gönderir ve durum/yanıtları assert ile denetler. Veritabanı :memory: olduğu için kendi data/app.sqlite dosyan etkilenmez. Projeye özgü testler tests/project.test.js içindedir.

```
npm ci
npm run typecheck
npm test
npm run build
```

Test kodundan örnek

```
const courses=await get('/courses?locale=en');assert.equal(courses[0].title,'Web
Development');assert.equal(courses[0].lessons[2].locale,'tr');
await call('/api/progress/l1','PUT',{completed:true},200);await
call('/api/progress/l1','PUT',{completed:true},200);
assert.equal((await
get('/courses?locale=tr'))[0].lessons.filter(x=>x.completed).length,1);
assert.equal((await call('/api/courses','GET',undefined,200,other))[0].lessons.filter(x=
>x.completed).length,0);
```

Tarayıcıda kontrol et

Bir başarılı işlem, bir hatalı girdi ve boş durum göster. 390 px mobil görünümde yatay taşmayı kontrol et. Sayfayı yenilediğinde kalıcı kayıtlar durmalı. Network panelinde isteğin gövdesini ve dönen HTTP kodunu oku; başarısız işlemi başarılıymış gibi göstermemelisin.

Doğrulamanın sınırı

API entegrasyon testleri, üretim derlemesi ve Vue tip/şablon kontrolü uygulandı. Bunlar yük testi, penetrasyon testi veya tüm tarayıcılarda kapsamlı uçtan uca test yerine geçmez.

05 / Çalışma ve GitHub planı

İlk günün somut hedefi

İngilizceye geç ve üçüncü dersin fallback uyarısını gör. Bir dersi iki kez tamamlayıp ilerlemenin yalnız bir ders arttığını doğrula.

Çalışma bloğu	Teslim edeceğin çıktı
1 · Kurulum / 30-60 dk	Çalışan ekran, örnek veri ve ilk başarılı API isteği
2 · Kod okuma / 45-90 dk	Formdan SQL sorgusuna kadar bir isteğin açıklaması
3 · Geliştirme / 60-120 dk	Bir iş kuralı veya arayüz davranışında kendi değişikliğin
4 · Doğrulama / 30-60 dk	İlgili test ve README değişikliği; yeni ekran görüntüsü

Süreler tahmindir; hata çözme ve yeni kavramlar daha uzun sürebilir. Bu plan hazır temel sürüm üzerinde çalışmak içindir. Projeyi sıfırdan ve üretim kalitesinde bir günde bitirme vaadi değildir.

İleri sürüm: dört gerçek geliştirme

- 1. Kurs ve ders editörü ekle
- 2. Kayıt/enrollment modeli ekle
- 3. Quiz ve başarı kriterleri ekle
- 4. Çeviri eksiklerini yöneten panel ekle

Anlamli commit sırası

```
git add .
git commit -m "chore: add multilingual-lms baseline"
# Kendi değişikliğini yaptıktan sonra:
git add client server tests README.md
git commit -m "feat: describe your actual improvement"
```

Yeni boş GitHub reposuna remote ekleyip push et. Burada gerçek hesap/repo adresi bulunmadığı için otomatik yükleme yapılmadı. node_modules, data ve .env dosyalarını hariç tut. Her gün bir gerçek ilerleme paylaş; commit sayısını artırmak için geçmiş üretme.

Mülakatta anlat

Çeviri bulunmadığında boş içerik yerine fallback kullanmanın ürün etkisi nedir? Ders silinirse ilerleme nasıl değişir?

Özel kurulum / ilerleme notu

Önce yerel sürümü tamamla. Sonraki aşamada PostgreSQL, migration, pagination ve gerçek kullanıcı yönetimini ihtiyaca göre ekle. Ortak kurulum ayrıntıları README.md ve başlangıç rehberindedir.